AIRI Team PITB | AI Internship Task 1

Traffic Object Detection Using YOLOv8

End-to-end computer vision project: data, annotation, training, evaluation, error analysis and demo

Submitted by: Muhammad Arslan Arif

Submitted to: Omar Farooq, Associate AI/ML Engineer

Programme: AIRI Team PITB, AI Internship (Individual task)

Date: 15 October, 2026

Project link: https://drive.google.com/drive/folders/1OutK2zlnlTy92yoI6arh8x-xc_M7AMEv?usp=sharing

1. Project Overview and Problem Definition

This project builds an object detection system that finds road vehicles in street-level traffic images and draws a bounding box, a class name and a confidence score on each one. I followed the full workflow of a real computer vision project: choosing the problem, collecting and cleaning data, reviewing annotations, training YOLOv8 on Google Colab, evaluating it, analysing its mistakes and running it on unseen images and video.

Question	Answer
What does the system detect?	Vehicles in traffic scenes: **car, motorcycle, bus, truck and three-wheeler** (auto-rickshaw / CNG).
Why is this useful?	Detecting and counting vehicles is the first step of traffic monitoring, congestion analysis and road-safety tools.
Where can it be applied?	Roadside cameras, junction monitoring, vehicle counting, and smart-city traffic dashboards.
Target classes	car, motorcycle, bus, truck, three-wheeler (5 classes).
Final output	A trained model (best.pt) that returns boxes, labels and confidence scores for images and video, plus a small web demo built with FastAPI.

Note on classes. Option A in the task lists car, motorcycle, person, bus and truck. The dataset I used has no person class, and three-wheelers (rickshaw / CNG) are very common in it, so I used **three-wheeler** as the fifth class instead of person.

2. Data Collection

Source. Public Kaggle traffic dataset: "Road Vehicle Images Dataset" by Ashfak Yeafi (https://www.kaggle.com/datasets/ashfakyeafi/road-vehicle-images-dataset), Bangladeshi road vehicle images with YOLO annotations. License: Open Database License (ODbL) for the database; see the Kaggle page for the contents terms. My first attempt used a dataset made of frames from a single night-time video. Its images were almost identical, so I replaced it with a dataset that has many different scenes, angles, distances and lighting conditions.

2.1 Merging 21 source classes into 5

The source dataset has 21 vehicle classes. To get a clean 2 to 5 class problem I merged similar vehicles:

New class	Source classes merged into it
car	car, suv, minivan, taxi, police car
motorcycle	motorbike, scooter
bus	bus, minibus
truck	truck, pickup, van
three-wheeler	rickshaw, CNG three-wheeler, auto rickshaw

Images that contained other classes (bicycle, ambulance, army vehicle, human hauler, wheelbarrow, garbage van) were left out, so that no visible vehicle stays unlabeled and teaches the model that vehicles are background.

2.2 Selecting the final 250 images

- I selected **250 images** from about 2,400 eligible ones, taking care that every class is well represented.
- To avoid near-duplicates, images from the same source series with neighbouring frame numbers were not selected together.
- The set contains daytime and evening scenes, near and far vehicles, crowded and empty roads, and partly hidden objects.

Class	Images containing it	Boxes (subset)	Train/val/test
car	134	452	70 / 20 / 10 %
motorcycle	82	132	70 / 20 / 10 %
bus	142	290	70 / 20 / 10 %
truck	139	294	70 / 20 / 10 %
three-wheeler	128	391	70 / 20 / 10 %

Final split: **175 training, 50 validation and 25 test images** (70 / 20 / 10). The validation set has 321 boxes and the test set 192 boxes. Motorcycle is the smallest class (132 boxes), which matters later in the error analysis.

3. Data Annotation

Tool: Roboflow (annotation review and export). Labels are in YOLO format, one .txt file per image, with one line per object:

```
<class_id> <x_center> <y_center> <width> <height>   (all values normalized between 0 and 1)
```

```
Example line: 0 0.512500 0.460000 0.210000 0.180000
```

Guidelines followed: tight box around every visible object of the five classes, one box per object, no box around a group, unclear objects not labeled, partly visible objects labeled only when the class is clear, and consistent class names.

3.1 Manual annotation and review

I manually reviewed **160 of the 250 images** in Roboflow. For each image I checked that every visible vehicle of the five classes had a box, that the boxes were tight, and that the class was correct, and I fixed the boxes that needed changes.

Review item	Count
Images reviewed manually	160 of 250
Boxes in the 250-image subset before review	1,559
Boxes after review (Roboflow Classes & Tags)	2051
Net change in boxes	492

Annotation problems found during error analysis include overlapping and incomplete ground-truth boxes in crowded scenes (for example Asraf_12 and Dipto_-57) and van-like vehicles labeled as truck because of my class merge.

4. Dataset Structure and data.yaml

The dataset follows the structure required by the task:

```
cv_project/  dataset/images/{train,val,test}  dataset/labels/{train,val,test}
dataset/data.yaml  |  notebooks/  outputs/{training_results,predictions,demo_results}
models/best.pt  report/  README.md  requirements.txt
```

data.yaml tells YOLO where the data is and what the classes are:

Key	Value used	Meaning
path	/content/drive/MyDrive/cv_project/dataset	Root folder of the dataset. The other paths are relative to it.
train	images/train	Images the model learns from (175).
val	images/val	Images used to check progress during training (50).
test	images/test	Unseen images used for the final check (25).
names	0 bus, 1 car, 2 motorcycle, 3 three-wheeler, 4 truck	Maps each class number in the label files to its name. Order comes from the Roboflow export.

5. Google Colab Setup and Model Training

The project runs on Google Colab with Google Drive mounted. Ultralytics was installed with pip, and the GPU was checked before training: **CUDA available = True, GPU = Tesla T4 (about 15 GB)**. The notebook shows this output.

I trained YOLOv8n from pretrained weights as the baseline and YOLOv8s as a second experiment, with identical settings and the same data split.

Setting	YOLOv8n (baseline)	YOLOv8s
Pretrained weights	yolov8n.pt	yolov8s.pt
Epochs	50	50
Image size	640	640
Batch size	8	8
Platform / GPU	Google Colab, Tesla T4	Google Colab, Tesla T4
Training time	4.0 minutes	4.4 minutes
Model file size	6.5 MB	22.5 MB
Saved output	models/best.pt (final model)	yolov8s_experiment/weights/best.pt

The final model is **YOLOv8n**, because its accuracy was about the same as YOLOv8s while the file is roughly three times smaller.

6. Model Evaluation

6.1 What the metrics mean

- **Precision:** of all objects the model detected, how many were correct. High precision means few wrong detections.
- **Recall:** of all real objects in the images, how many the model found. High recall means few missed objects.
- **mAP@0.5:** overall detection quality, counting a box as correct when it overlaps the true box by at least 50% (IoU 0.5), averaged over classes.
- **mAP@0.5:0.95:** the same, averaged over stricter overlap levels from 0.5 to 0.95. It rewards tight, accurate boxes.

6.2 Results

Model	Split	Precision	Recall	mAP@0.5	mAP@0.5:0.95
YOLOv8n (baseline)	val	0.545	0.426	0.424	0.261
YOLOv8n (baseline)	test	0.586	0.458	0.476	0.274
YOLOv8s	val	0.602	0.393	0.417	0.261
YOLOv8s	test	0.594	0.431	0.463	0.263

The test set has only 25 images (192 boxes), so small differences between the models are not reliable. YOLOv8s gave higher precision but lower recall, and no real gain in mAP, which suggests the limit is the amount and quality of data, not the model size.

6.3 Per-class performance (YOLOv8n, validation set)

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
bus	0.490	0.369	0.424	0.284
car	0.539	0.584	0.513	0.351
motorcycle	0.572	0.383	0.382	0.161
three-wheeler	0.567	0.287	0.336	0.203
truck	0.539	0.449	0.464	0.312
all	0.541	0.415	0.424	0.262

- **Best class: car** (mAP@0.5 = 0.513, recall 0.584). It has the most examples (452 boxes).
- **Weakest classes: three-wheeler** (lowest recall, 0.287, many missed) and **motorcycle** (lowest mAP@0.5:0.95, 0.161, boxes are not tight). Motorcycle has the fewest training boxes and the objects are small.

6.4 Confusion matrix and loss curves

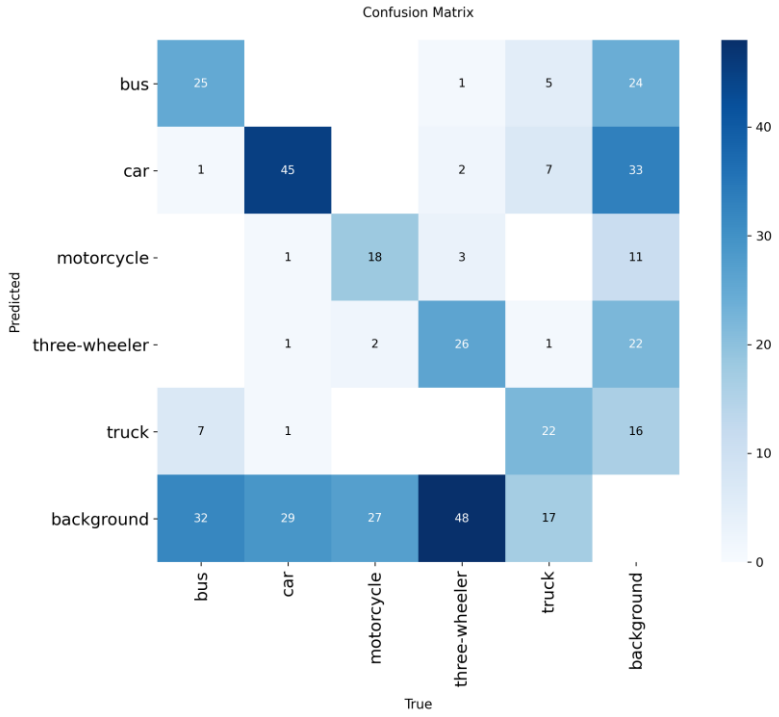


Figure 1. Confusion matrix of YOLOv8n on the validation set (confidence 0.25). Rows are predicted classes, columns are true classes.

The matrix at confidence 0.25 shows that most class mix-ups involve **truck**: for example, objects labeled truck were predicted as car in 7 cases. A large part of the errors are objects the model missed or boxes placed where the labels show nothing (the background column).

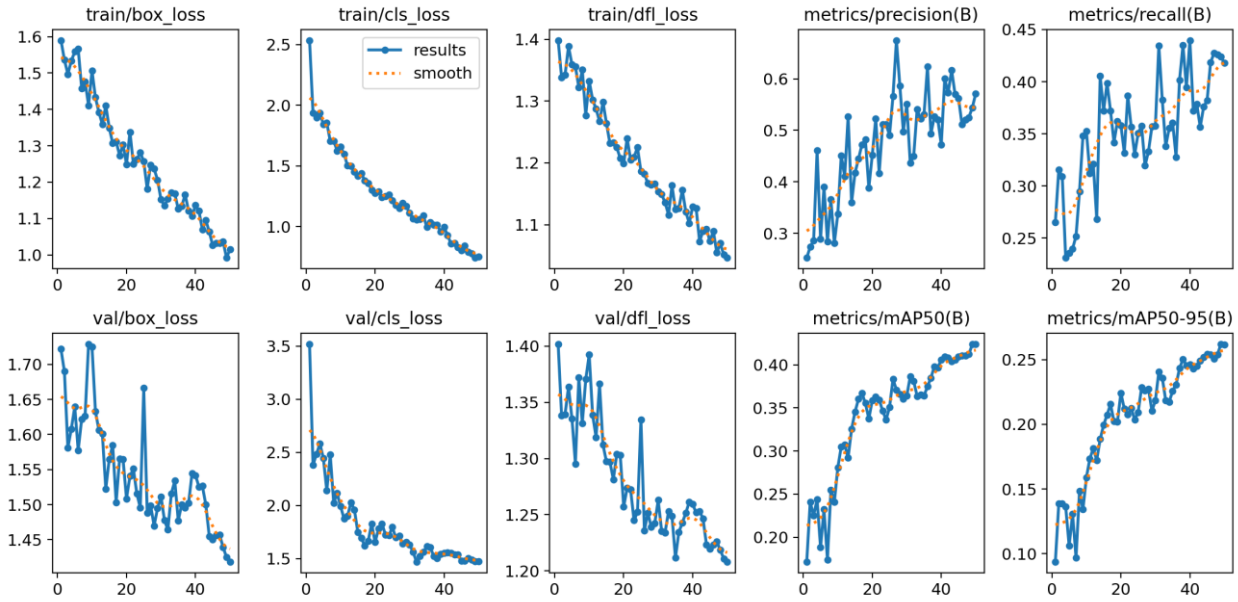


Figure 2. Training and validation curves over 50 epochs: losses fall steadily, while precision, recall and mAP rise.

The training and validation losses go down over the 50 epochs, so the model learned from the data without obvious overfitting, but the final accuracy stays limited by the small dataset.

7. Inference on New Data

7.1 Unseen test images

The final model was run on the 25 unseen test images with confidence threshold 0.35. Each output image shows the bounding boxes, predicted labels and confidence scores. The results are saved in outputs/predictions/test_predictions.



Figure 3. Predictions of the final model on 20 of the 25 unseen test images (confidence threshold 0.35).

7.2 Video inference

I ran the model on a 28-second street-level traffic clip (resized from 4K to 1280x720, 681 frames). The annotated output was saved as an 8.6 MB mp4 file in outputs/demo_results/video_demo. Video source: Pexels, https://www.pexels.com/video/32665228/ (free stock video, Pexels License: free to use, attribution not required). Creator: Riaz Uddin.

Looking at frames from the output video, the model detects buses and cars well and steadily: bus confidence is mostly 0.88 to 0.95, and scooters and motorcycles are usually found. Mistakes are visible too: a yellow taxi was called truck (0.67) in one frame, crowded moments produce duplicate overlapping car boxes, and short-lived low-confidence three-wheeler boxes appear on pedestrians and small background objects although no three-wheeler is in this clip. Small and distant vehicles are often missed, and the model has no person class, so pedestrians sometimes receive vehicle boxes.



Figure 4. Four frames from the output video (seconds 1.3, 7.5, 13.8 and 26.3): buses and cars detected well; a yellow taxi labeled truck and duplicate car boxes in the third frame.

7.3 Web demo (FastAPI)

I built a small FastAPI application. The endpoint /predict accepts an image and returns the detections as JSON (class, confidence, box), /predict-image returns the image with boxes drawn, and the home page offers a simple upload interface with a confidence slider and per-class counts. It runs locally with: python -m uvicorn main:app --port 8000.

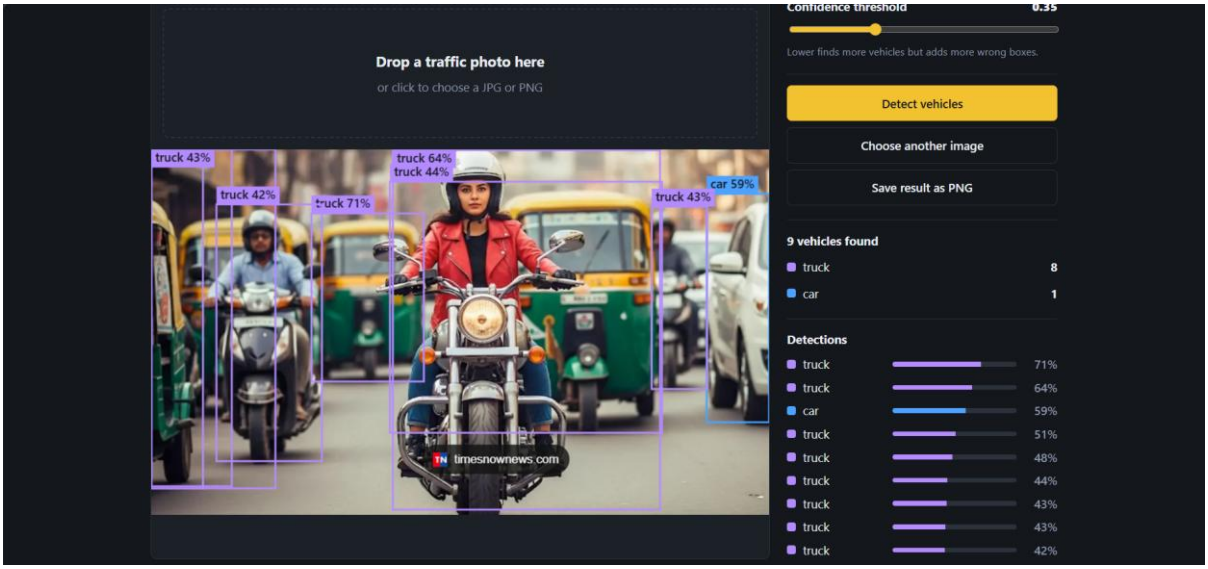


Figure 5. Web demo running locally. The close-up test photo shows the main weakness of the model (see Section 8).

8. Error Analysis

I reviewed 15 weak or incorrect predictions by comparing ground truth (green) with predictions (red), using a script that lists the images with the most missed, wrong-class and extra detections. Reasons below are my observations from the images.

Image	Actual object	Model prediction	Error type	Possible reason
Navid_62	bus	bus 0.21 (below 0.35)	False negative (low confidence)	Confidence under the threshold; bus/truck confusion
Navid_62	motorcycle	not detected (even at 0.1)	False negative	Small and partly visible; few motorcycle examples
Pias-191	motorcycle (small)	not detected	False negative, small object	Far away, low light, only tail lights visible
Pias-191	car (behind another)	not detected	False negative	Partly hidden behind a white car
Pias-191	vehicles on the right	several overlapping car boxes	False positive, duplicates	Vehicles very close together; source labels overlap too
212	58 vehicles	11 detected	False negative (dense scene)	Very crowded; small, overlapping vehicles
Asraf_41	yellow bus, car at edge	not detected	False negative	Partly hidden or cut by the image edge
Dipto_-57	truck (van-like)	car 0.87	Wrong class	Looks like a minivan; vans were merged into truck
Dipto_-57	no label	boxes on a cart and railing	False positive / annotation mistake	Cart-like object; ground-truth label missing
Asraf_12	bus (large, near)	not detected	False negative	Cut by the image edge and partly hidden
Asraf_12	bus, truck, three-wheeler	car 0.47, car 0.65	Wrong class + annotation mistake	Overlapping ground-truth boxes in a crowded scene
Numan_-383	bus	truck 0.75	Wrong class	Small bus looks like a truck from far away
Numan_-383	green vehicle	bus 0.47 and three-wheeler 0.48	Wrong class (two boxes)	Class confusion; few similar training examples
Numan_-383	small distant buses	not detected	False negative, small object	Very small and far away
Close-up photo (external)	motorcycles, three-wheelers	truck (8 boxes)	Wrong class + poor boxes	Close-up large objects not seen in training; truck class too broad

8.1 Examples



Figure 6. Dipto_-57: van labeled truck, predicted car; extra boxes on a rickshaw cart (left: ground truth, right: prediction).

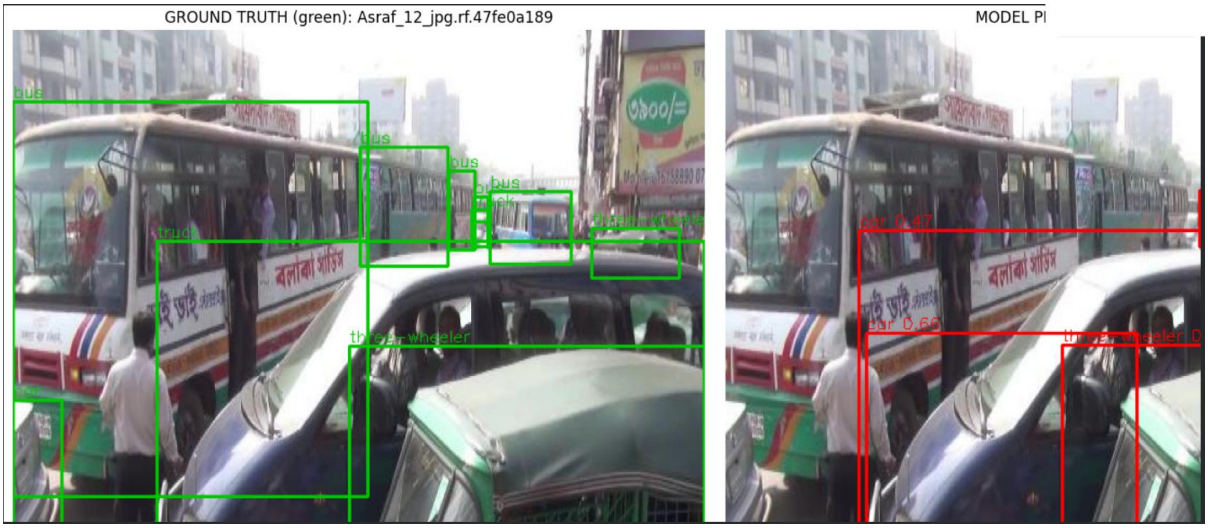


Figure 7. Asraf_12: large near bus missed; crowded scene with overlapping ground-truth labels.

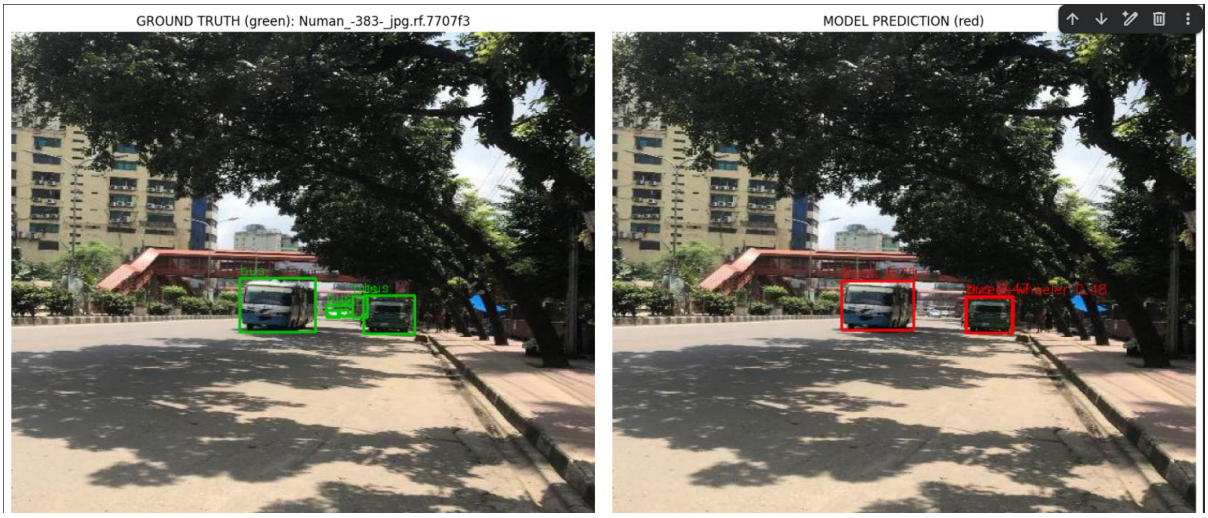


Figure 8. Numan_-383: bus predicted as truck; distant small buses not detected.



Figure 9. Pias-191: evening light, small motorcycles missed, overlapping boxes on the right.

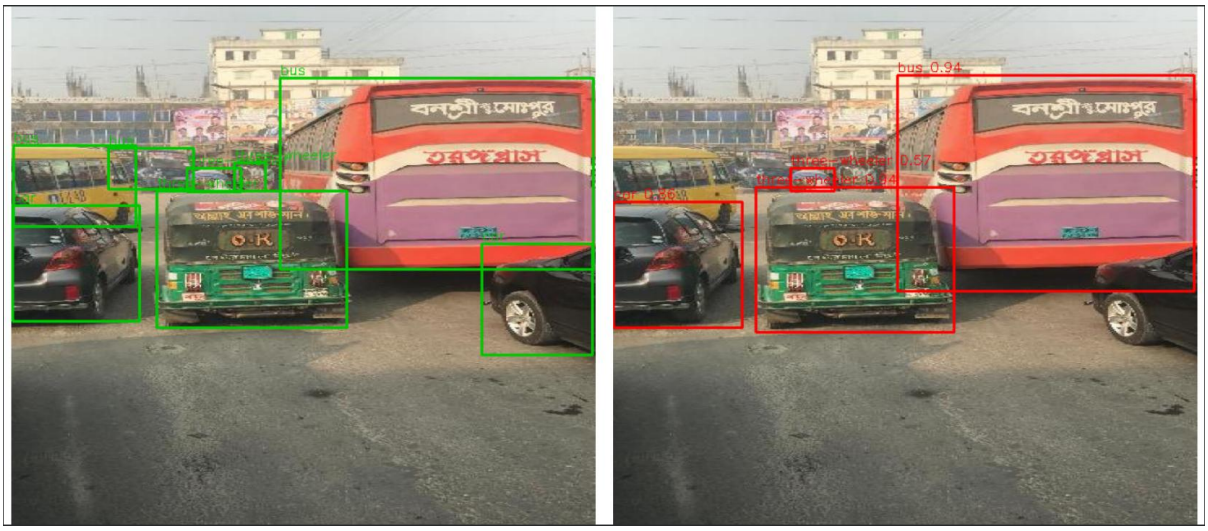


Figure 10. Partly hidden and edge-cut vehicles are missed.

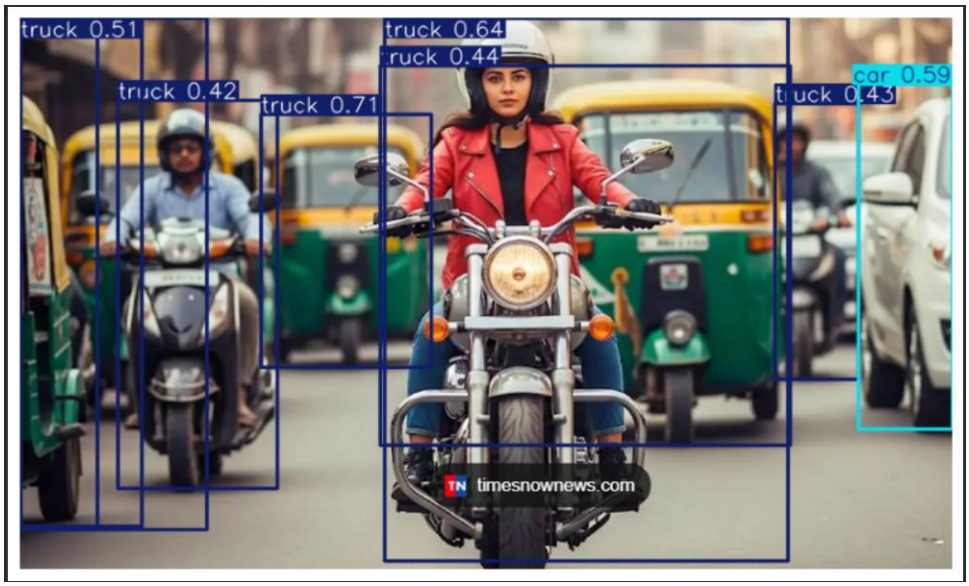


Figure 11. Close-up test photo (external image from a news website, used only for testing): motorcycles and three-wheelers are wrongly predicted as truck, with oversized boxes.

8.2 Error types found

- **False negatives:** small, distant, hidden or edge-cut vehicles (Pias-191, Asraf_12, Numan_-383, image 212).
- **False positives:** duplicate boxes in crowded areas, and boxes on objects that have no label (Dipto_-57).
- **Wrong class:** truck confused with car and bus; in a close-up photo motorcycles and three-wheelers were called truck.
- **Poor bounding boxes:** oversized boxes in the close-up photo.
- **Small-object problems:** far-away motorcycles and buses.
- **Poor lighting:** the evening scene Pias-191.
- **Annotation mistakes:** overlapping or missing ground-truth boxes (Asraf_12, Dipto_-57), and van-like vehicles labeled truck because of my class merge.
- **Threshold effect:** the Navid_62 bus was found at 0.21 but hidden by the 0.35 threshold. A lower threshold finds more objects but adds more wrong boxes.

8.3 Suggested improvements

- **Fix the class merge.** Move van and pickup out of truck (into car, or remove them) so truck means real trucks only.
- **Add more and harder data**, especially close-up views, small and dark scenes, and partly hidden vehicles, with extra images for motorcycle and three-wheeler. Adding my own annotated photos is the quickest way.
- **Clean the labels.** Fix overlapping, missing and wrongly classed boxes in Roboflow before retraining.
- **Balance the classes** so motorcycle and three-wheeler have as many examples as car.
- **Then try a larger model or more epochs.** YOLOv8s alone did not help, so this only makes sense after the data is improved.

9. What I Learned

- How to prepare a computer vision dataset: choosing classes, merging similar ones, selecting varied images and splitting them 70 / 20 / 10.
- The YOLO label format and how data.yaml connects images, labels and class names.
- How to review and correct bounding boxes in Roboflow, and how many label problems a public dataset can contain.
- How to train YOLOv8 on Google Colab with a GPU and save the best checkpoint.
- How to read precision, recall, mAP and the confusion matrix, and why a small test set gives unstable numbers.
- That a bigger model did not fix the problem, and that data quality and class design matter more than the algorithm.
- How to turn a model into a simple FastAPI demo that others can try.

10. Future Improvements

- Collect a larger, more balanced dataset (400+ images) with close-up, small-object and low-light examples.
- Redefine the truck class and review all labels.
- Train a larger YOLO model on the improved data and compare it with this baseline.
- Deploy the demo online, and test on real-time video or CCTV streams.

11. Conclusion

The project covers the full pipeline from data to a working demo. The final YOLOv8n model reaches mAP@0.5 of 0.424 on the validation set and 0.476 on the test set. It works reasonably on scenes similar to the training data, but fails on close-up views and confuses truck with other vehicles. Error analysis shows that the main causes are the small dataset, the broad truck class and label problems, which gives a clear plan for the next version.

Project Links and Credits

Item	Details
Google Drive / GitHub	https://drive.google.com/drive/folders/1OutK2zlnlTy92yoI6arh8x-xc_M7AMEv?usp=sharing
Dataset source and license	"Road Vehicle Images Dataset" by Ashfak Yeafi (https://www.kaggle.com/datasets/ashfakyeafi/road-vehicle-images-dataset), Bangladeshi road vehicle images with YOLO annotations. License: Open Database License (ODbL) for the database; see the Kaggle page for the contents terms
Video credit	Pexels, https://www.pexels.com/video/32665228/ (free stock video, Pexels License: free to use, attribution not required). Creator: Riaz Uddin.
Tools	Python, Google Colab, Google Drive, YOLOv8, Ultralytics, OpenCV, Roboflow, Matplotlib, Pandas, FastAPI